# INDEX

NAME: T. Srinivasa Reddy STD.: btech SEC.: 89 ROLL NO.: A499 SUB.: DSA (observation).

| S. No. | Date | Title | Page No. | Teacher's Sign / Remarks |
|---|---|---|---|---|
| 1. | 28/7/26 | Arrays of structure creation, insertion deletion, traversal, Sorting. | | |
| 2. | 5/8/26 | Student details. | | |
| 3. | 12/8/26 | Display value and address using pointers | | |
| 4. | 19/8/26 | C program for building a dynamic Employee Record system using structure pointers. | | |
| 5. | 26/8/26 | stack using Array. | | |
| 6. | 2/9/26 | Simple Queue | | |
| 7. | 9/9/26 | Single linked list. | | |
| 8. | 16/9/26 | Circular Queue | | |
| 9. | 7/10/26 | Binary search tree. | | |
| 10. | 14/10/26 | graph search algorithm (BFS & DFS) | | |
| 11. | 12/10/26 | Implementation of Bubble, Selection and Heap sorts. | | |
| 12. | 21/10/26 | Implementation of Hashing with collision Resolution using Linear probing | | |

# ① Basic Array operations

29/7/2026

1) Display
2) Insertion
3) Deletion
4) searching.
5) Modify.

Ex-1:- Print All Array Elements forward.

Aim:- To write a program to display all elements of an array.

Algorithm:-

1. start
2. Read array size 'n' and array elements arr[]
3. Loop 'i' from 0 to n-1
4. Print arr[i]
5. stop.

Program:-

```
#include <stdio.h>
int main()
{
    int arr[] = {10, 20, 30, 40, 50};
    int n = 5;
    for (int i = 0; i<n; i++)
{
    Printf("%d", arr[i]);
    }
Printf("\n");
return 0;
```

Sample output:-

10 20 30 40 50

Result:-

The program for array displaying was executed and verified.

Ex-2:-

Aim:-

To write a c program to display array elements in reverse order.

Algorithm:-

1. start
2. Read array size n and array elements arr[]
3. Loop i from n-1 to 0:
4. Print arr[i]
5. stop.

Program:-

```
#include <stdio.h>
 int main()
  {
    int arr[] = {12, 24, 36, 48, 60};
     int n=5;
     Printf("Reverse order:");
  for (int i=n-1; i>=0; i--).
   {
     Printf("%d", arr[i]);
   }
  Printf("\n");
   return 0;
  }
```

Sample output:-

Reverse order: 60 48 36 24 12

Result:-

The program for displaying reverse array is successful.

Ex-3:-

Aim:-

To write a C program to insert a new element in an array

Algorithm:-

1. Start
2. Read values, size n, elements arr[] and insertion index pos.
3. shift elements from "n-1" down to pos -
4. arr[i+1] = arr[i]
5. Insert arr[pos] = val.
6. set n = n+1
7. stop.

Program:-

```
#include <stdio.h>
int main()
{ int arr[10] = { 10, 20, 40, 50 };
  int n=4, val = 30, pos = 2;
for (int i = n-1; i >= pos; i--)
  { arr[i+1] = arr[i];
  }
arr[pos] = val;
n++;
Printf ("Array after insertion:");
for (int i = 0; i<n; i++)
{ printf ("%d", arr[i]);
}
printf("\n");
return 0;
}
```

Sample output:-

Array after insertion: 10 20 30 40 50

Result:-

Insertion at a specific index was executed successfully.

Aim:-

To write a program to insert newelemet at the beginning.

Algorithm:-

1. Start
2. Read values
3. arr[i+1] = arr[i]
4. Set arr[0] = val
5. Set n=n+1
6. stop.

Program:-

```
#include <stdio.h>
 int main()
     {
      int arr[10] = {2, 3, 4, 5};
       int n= 4, val=1;
     for (int i = n-1; i>=0; i--)
     { arr[i+1]=arr[i];
         }
     arr[0] = val;
      n++;
    Printf("Array after inserting at beginning:");
    for (int i =0; i<n ; i++)
      { printf("%d ", arr[i]);
                    }
       Printf("\n");
       return 0;
  }
```

Sample output:-

Array after inserting at beginning: 1 2 3 4 5

Result:-

Insertion at the beginning of the array was executed succesfully.

Ex-5:-

Aim:-

To write a C program to delete an array from index in a array.

Algorithm:-

1. Start.
2. Read size, arr[] and pos.
3. Shift elements from pos+1 to n-1
4. arr[i-1] = arr[i]
5. set n=n-1.
6. stop.

Program:-

```
#include <stdio.h>
  int main()
    {
     int arr[] = { 10, 20, 99, 30, 40};
      int n=5, pos= 2;
     for (int i = pos+1 ; i<n; i++)
     { arr[i-1] = arr[i];
              }
        n--;
     Printf ("Array after deletion :");
     for (int i=0; i<n; i++)
        Printf ("%od", arr[i]);
        Printf ("\n");

        return 0;
     }
```

Sample output:-

Array after deletion : 10 20 30 40

Result:-

Deletion from a specified index was executed successfully.

Ex-6:-

Aim:-

To write a C program to delete the last element of an array

Algorithm:-

1. Start
2. Read n and arr[].
3. Decrement n=n-1
4. Stop.

Program:-

```
#include <stdio.h>
 int main()
    {
      int arr[] = { 5, 10, 15, 20} ;
       int n=4;
       n--;
      Printf ("Array after deleting last element: ");
      for (int i=0; i<n; i++)
      {
        Printf ("%d", arr[i]);
                }
      Printf ("\n");
        return 0;
      }
```

Sample output:-

Array after deleting last element : 5 10 15

Result:-

Deletion of the last elemed was executed successfully.

Ex. No: 9

Aim:
To write a C program to search for a key element in an array.

Algorithm:
1. Start
2. Read n, arr[], key.
3. Loop i from 0 to n-1;
4. If arr[i] == key print found at index i and stop.
5. else print target not found after the loop.
6. Stop.

Program:
```
#include <stdio.h>
int main()
{ int arr[] = { 15, 25, 35, 45, 55};
  int n=5 ; key=35 , found=-1 ;
for (int i=0 ; i<n; i++)
  { if (arr[i] == key)
    { found = i;
      break;
    }
  }
if (found != -1)
    printf ("Element %d found at index %d.\n", key, found);
else
    printf("Element not found.\n");
return 0;
}
```

Sample output:
Element 35 found at index 2

Result:
Linear search was executed successfully.

Ex - 8:-

Aim:-

To write a C program to search and count how many times a value appears in an array

Algorithm:-

1. Start
2. Read n, arr[], key, count = 0.
3. Loop i from 0 to n-1;
4. If arr[i] == key, increment count = count + 1.
5. print count
6. stop.

Program:-

```
#include <stdio.h>
  int main()
    {
      int arr[] = { 5, 2, 5, 8, 5, 9 };
       int n = 6, key = 5, count = 0;
      for (int i=0; i<n; i++)
        { if (arr[i] == key)
          {
            count ++ ;
          }
        }
  Printf("%d appeared %d times. \n", key, count);
    return 0;
  }
```

Sample output:-

Element 5 appeared 3 times.

Result

Counting occurrences via searching was executed successfully

Exp 7

Aim:
To write a C program to modify the certain element in an array.

Algorithm:

1. start
2. Read n, arr[], pos, newValue
3. arr[pos] = newValue
4. stop

Program:

```
#include <stdio.h>
int main()
{
    int arr[] = {10, 20, 99, 40};
    int pos = 2, newValue = 30;
    arr[pos] = newValue;
    printf("Array after update:");
    for (int i=0; i<4; i++)
    {
        printf("%d", arr[i]);
    }
    printf("\n");
    return 0;
}
```

Sample output:

Array after update: 10 20 30 40

Result:

Array element update was executed successfully.

Aim:-

To write a c program to scale all elements by a factor of 2

Algorithm:-

1. Start
2. Read n, arr[]
3. Loop 'i' from 0 to n-1:
4. Set arr[i] = arr[i] * 2.
5. Display modified elements.
6. stop.

Program:-

```
#include <stdio.h>
int main()
    {
      int arr[] = {1, 2, 3, 4, 5};
       int n=5;
      for (int i=0; i<n; i++)
        { arr[i] = arr[i] * 2;
        }
      Printf("Array after doubling values:");
    for(int i=0; i<n; i++)
        { printf("%d", arr[i]);
                    }
          Printf("\n");
         return 0;
    }
```

Sample output:-

Array after doubling values : 2 4 6 8 10

Result:-

(3) C program for Building a Dynamic Employee Record system using structure pointers.

Ans:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX_EMPLOYEES 100

struct Employee
{ int id;
  char name[50];
  float salary;
  };
struct Employee *emp[MAX_EMPLOYEES];

int count = 0;
void addEmployee()
{ if (count >= MAX_EMPLOYEES)
{ Printf("Employee list is full \n");
  return;
}
struct Employee*e = (struct Employee*)
                    malloc(sizeof(struct employe));

Printf("Enter ID: ");
scanf("%d", &e->id);
Printf("Enter Name: ");
scanf("%s", e->name);
Printf("Enter salary: ");
scanf("%F", &e->salary);

emp[count] = e;
count++;
```

```
Printf ("Employee added ! \n");
 }
void displayEmployee()
 {
   if (count == 0)
    { Printf("No employee to show \n");
    return;
    }
   Printf ("\nID \t Name \t Salary \n");
for (int i=0; i< count; i++)
   {
     Printf ("%d \t %s \t %.2f \n", emp[i] -> id,
                                    emp[i] -> name, emp[i] -> sal
   }
Printf("\n");
}
void searchEmployee()
     { int id;
       Printf ("Enter ID to search : ");
       scanf("%d", &id);
  for (int i=0 ; i< count ; i++)
        { if (emp[i] -> id == id)
   { printf("Found %s Salary : %.2f \n",
                                    emp[i] -> name, emp[i] -> sal
    return;
   }
  }
Printf("Employee not found. \n");
   }
   void deleteEmployee()
     { int id;
       Printf("Enter ID to delete:");
       Scanf("%d", &id);
```

```
for (int i = 0; i < count; i++)
{
if (emp[i]->id == id)
{
free(emp[i]);
emp[i] = emp[count-1];
count--;
printf("Employee deleted.\n");
return;
}
}
printf("Employee not found.\n");
}
void free_all()
{
for (int i=0; i<count; i++)
{
free(emp[i]);
}
count = 0;
}
int main() {
    int choice;
    do {
printf("1. Add 2. Display 3. Search 4. Delete 5. Exit\n");
printf("Choice: ");
scanf("%d", &choice);
switch (choice) {
case 1: addEmployee(); break;
case 2: displayEmployee(); break;
case 3: searchEmployee(); break;
case 4: deleteEmployee(); break;
case 5: freeAll(); printf("Bye!\n"); break;
default: printf("Invalid choice\n");
}
} while (choice != 5);
return 0;
}
```

Output :-

1. Add 2. Display 3. Search 4. Delete 5. Exit.

choice : 2

No employee to show.

1. Add 2. Display 3. Search 4. Delete 5. Exit.

choice 1

Enter ID: 101

Enter name: aaa

Enter Salary : 10000

Employee added !

1. Add 2. Display 3. Search 4. Delete 5. Exit.

choice 1

Enter ID : 102

Enter name : bbb.

Enter salary : 20000

Employee added !

1. Add 2. Display 3. Search 4. Delete 5 Exit.

Choice 1

Enter ID : 103

Enter name : ccc

Enter salary : 30000

Employee added !

1. Add 2. Display 3- Search 4. Delete 5. Exit.

Choice 1

Enter ID : 104

Enter name : ddd

Enter salary : 40000

Employee added !

1. Add 2. Display 3. Search 4. Delete 5. Exit.

choice: 2

| ID | Name | Salary. |
|---|---|---|
| 101 | aaa | 10000.00 |
| 102 | bbb | 20000.00 |
| 103 | ccc | 30000.00 |
| 104 | ddd | 40000.00 |

1. Add 2. Display 3. Search 4. Delete 5. Exit.

choice: 3

Enter ID to search : 102

Found :- Name: bbb ; Salary: 20000.00.

1. Add 2. Display 3. Search 4. Delete 5. Exit.

choice: 4

Enter ID to delete 04

Employee deleted!

1. Add 2. Display 3. Search 4. Delete 5. Exit.

choice: 2.

| ID | Name | Salary. |
|---|---|---|
| 101 | aaa | 10000.00 |
| 102 | bbb | 20000.00 |
| 103 | ccc | 30000.00 |

1. Add 2. Display 3. Search 4. Delete 5. Exit.

choice: 5

Bye!!

```
#include <stdio.h>
#define max 5
int stack[max];
int top = -1;
void push(int value)
{
if (top == max-1)
{
printf("Stack Overflow\n");
}
else
{
top++;
stack[top] = value;
printf("%d inserted into stack\n", value);
}
}
int main()
{
push(10);
push(20);
push(30);
printf("Stack elements are:\n");
for (int i=top; i>=0; i--)
{
printf("%d\n", stack[i]);
}
return 0;
}
```

output:-

Enter the number of elements to push: 3

Enter values : 10

10 pushed into stack

Enter the values : 20

20 pushed into stack

Enter value : 30

30 pushed into stack.

pop().

Programme:-

```
#include <stdio.h>
#define MAX 5.
int stack [MAX] = [10, 20, 30] ;
int top= 2;
void pop()
{
if (top = = -1)
{printf ("stack underflow \n");
}
else
{printf ("%d popped from stack \n", stack[top]);
top--;
}
}
int main()
{ printf ("stack before pop : \n");
for (int i= top; i>=0; i--)
{ printf ("%d \n", stack [i]);
}
pop();
printf ("\n stack after pop : \n");
for(int i = top ; i>=0; i--)
{printf ("%d \n", stack[i]);
}
return 0;
}
```

output:-

```
stack before pop
30
20
10
Deleted element : 30
stack after pop:
20
10.
```

3. Peek()

Programme :-

```
#include <stdio.h>
#define MAX 5
int stack[MAX] = {10, 20, 30};
int top = 2;
void peek()
{ if (top == -1)
  { printf("stack is empty \n");
  }else
  { Print("Top element is : %d \n", stack[top]);
  }
}
int main()
{
  printf("stack element are : \n");
  for(int i = top; i >= 0 ; i--)
  { printf("%d \n", stack[i]);
  }
  peek();
  return 0;
}
```

output:

stack elements:

30
20
10

Top element:

30

(9) Simple queue:-

Program:-

```
#include <stdio.h>
#define MAX 5
int queue [MAX];
int front = -1;
int rear = -1;
// Add element
void enqueue (int value)
{ if (rear == MAX-1)
{ printf("Queue overf \n");
  return 0; }
 if (front == -1)
  front = 0;
  rear + t;
 queue [rear] = value;
printf ("%d in sorted \n", value);
}
// remove element
void dequeue[]
  { if (front == -1 || front > rear)
  { printf ("queue underflow\n");
  return; }
printf-("%d remove \n", queue [front]);
front ++;
}
// display queue
void display ()
{ if (front == -1 || front > rear)
  { printf("Quee is empty \n");
      return;
      }
```

```
    printf("Queue :");
for(int i = front ; i <= rear ; i++)
    printf("%d", queue[i]);
    printf("\n");
}
int main()
{
    enqueue(10);
    enqueue(20);
    enqueue(30);
    display();
    dequeue();
    display();
    return 0;
}
```

output:-

10 inserted into Queue
20 inserted into Queue
30 inserted into Queue.
Queue element : 10 20 30
10 deleted from queue
Queue element : 20 30
40 insert into Queue
Queue element 20 30 40.

(2) Postfix( ):-

Program:-

```
#include <stdio.h>
#include <ctype.h>
#define MAX 100
int stack [MAX];
int top = -1;
// push
void push (int value)
{
  stack [++top] = value;
}

// pop.
int pop ()
{ return stack [top--];
}
// Evaluate postfix
int Evaluate postfix (char postfix [])
{ int i;
  char c;
  int a, b;
  for (i=0;
```

(5) Conversion of infix to postfix

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>
#define MAX 100
char stack[MAX];
int top=-1;
// push operation
void push (char item)
{ stack[++top] = item;
}
// pop operation
char pop()
{ return
  stack[top--];
}
// return precedence of operators
int precedence (char op)
{ switch (op)
  { case '^' : return 3;
    case '*'
    case '/' : return 2;
    case '+'
    case '-' : return 1;
    default : return 0;
  }
}
int main()
{ infix[max], postfix[MAX];
```

```
int i, j = 0;
Printf (" Enter Infix Expression: ");
 Scanf(" %s", infix);
  for(i=0; infix[i]!= '\0', i++)
{ char ch = infix[i];
    // operand
  if (isalnum(ch))
  { postfix[j++]=ch;
  }
   // Left paranthesis
  else if (ch == '(') {
      Push (ch);
  }
   // Right paranthesis
else if (ch == ')' ) {
    while (top != -1 && stack[top] != '(') {
      Post fix [j++] = pop();
  } pop (); // Remove '('

  } // operator.
     else {
  while (top != -1 && precedence (stack[top] >= precedence (ch))
  { postfix [j++] = pop();
      } push (ch);
    }
    // pop remaining operators
  while (top != -1) {
      postfix [j++] = pop();
    } postfix[j] = '\0';
  Printf ("postfix Expression: %s \n", postfix);
   return 0;
  }
```

Output:-

Enter Infix Expression

A+B* C

(6) Evaluate Postfix :-

Program :-

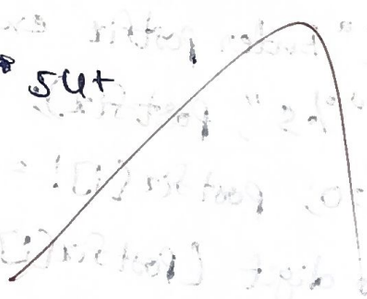
```
#include <stdio.h>
#include <ctype.h>
#define MAX 100
int stack[MAX];
int top = -1;
void push(int value)
{
stack[++top] = value;
}
int pop()
{
return stack[top--];
}
int main()
{
char postfix[MAX];
int i, a, b, result;
Printf("Enter postfix expression:");
Scanf("%s", postfix);
for(i=0; postfix[i] != '\0'; i++)
{ if(isdigit(postfix[i]))
{ push(postfix[i] - '0');
}
else
{
b = pop();
a = pop();
switch(postfix[i])
{ case '+':
result = a+b;
break;
```

```
case '-':
result = a-b;
break;
case '*':
result = a* b;
break;
case '/':
result = a/b;
break;
}
Push (result);
}
}
Prutf ("Result = %d \n", Pop());
return 0;
}
```

output:-

Enter postfix expression: 23* 54+

Result = 26.



## 7) Single Linked List

Program:-

```
#include <stdio.h>
#include <stdlib.h>
  struct Node
  { int data;
   struct node* next;
  };
struct node * head = NULL;
// Insert at beginning.
  void insert (int value)
{ struct Node * newNode;
newNode = (struct Node*)malloc(sizeof (struct node));
newNode -> data = value;
   newNode -> next = head;
     head = newNode;
}
// Delete from beginning
void deleteNode ()
  { struct Node* temp;
   if (head == NULL)
   { printf ("List is empty");
    return;
    }
   temp = head;
   head = head -> next;
  free (temp);
  printf ("Node deleted successfully \n");
}
// display the linked list
    void display ()
        {
     struct Node * temp = head;
```

```
if (head == NULL)
  {
  Printf ("List is empty \n");
return;
}
Printf ("Linked List:");
while (temp! = NULL)
{ Printf ("%d → ", temp → data);
  temp = temp → next;
} Printf ("NULL \n");
} int main()
    {
    int choice, value;
  while (1)
      {
    Printf ("\n--- Singly Linked List ---\n");
     Printf ("1. Sigert \n");
    Printf ("2. Delete \n");
   Printf ("3. Display \n");
   Printf ("4. Exit \n");
  Printf ("Enter your choice: ");
Scanf ("%d", &choice);
  Switch (choice) {
  case 1:
   Printf ("Enter value:");
   scanf (".%d", &value);
    insert (value);
     break;
    Case 2:
    delete node ();
      break;
      case 3;
      display ();
```

```
case 4:
 exit(0);

 default:
 Printf("Invalid choice \n");
 }

 }
 return 0;
 }
```

output:-

--- Single Linked List ---

1. Insert
2. Delete
3. Display
4. Exit.

Enter your choice : 1

Enter value : 10

Enter your choice : 1

Enter value : 20

Enter your choice : 3

Linked List : 20 → 10 → NULL

(8) Priority Queue using Unsorted array.

Program:

```
#include <stdio.h>
#define MAX 100
 int pq[MAX];
   int size =0;
  // Insert element
 void insert (int value)
{ if (size == NULL)
{ printf ("Priority Queue is Full \n");
 return;
} pq[size] = value;
  size ++;
  printf ("%d inserted \n", value);
}
// delete highest priority element
void delete Highest priority()
{ if (size == 0)
{ printf (" Priority Queue is Empty \n");
 return;
 }
 int i, max Index = 0;
 // Find highest priority element
for(i=1; i<size; i++)
{ if (pq[i] > pq[max Index])
 {
 max Index = i;
    }
```

```
Printf("%d deleted \n", pq[maxIndex]);
// shift elements
for(i= maxIndex; i< size -1; i++)
{
pq[i] = pq[i+1];
}
size--;
}
// display queue
Void display()
{ int i;
if (size==0)
{ Printf("priority Queue is Empty \n");
return;
}
Printf("Priority Queue:");
for(i=0; i<size; i++)
{ Printf("%d", pq[i]);
}
Printf("\n");
}
int main()
{ int choice, value;
while(1)
{
Printf("\n --- priority Queue --- \n");
Printf("1. Insert \n");
Printf("2. Delete Highest priority \n");
Printf("3. Display \n");
Printf("4. Exit \n");
```

Enter choice: 1

Enter value: 10

10 Inserted

Enter your choice: 1

Enter value: 50

50 inserted

Enter your choice: 1

Enter value: 20

20 inserted

Enter your choice: 3

Priority Queue: 30 10 50 20

Enter your choice: 2

50 deleted

Enter your choice: 3

Priority Queue: 30 10 20

Enter your choice: 4

Exit

Result:-

Thus the program was executed successfully.

(9) Circular Queue

20/9/2026

Programme

```
#include <stdio.h>
#include <stdlib.h>
struct Node
{
  int data;
   struct Node *next;
};
struct Node *front = NULL, *rear = NULL;
void enqueue (int value) {
   ~~Struct Node *front = NULL;~~
struct Node *newnode = (struct Node*) malloc(sizeof(struct Node));
newnode -> data = value;
  if (front == NULL)
  { front = rear = newNode;
    newNode -> next = front;
  } else
    {
     rear -> next = newNode;
     rear = newNode;
      rear -> next = front;
  } Printf("%d enqueued: \n", value);
   } void dequeue()
      { if (front == NULL);
          { printf("Queue underflow \n");
            return;
          }
       if (front == rear)
       { printf("%d dequeued. \n", front->data);
```

```
    free (front);
    front = rear = NULL;
    return;
}
struct Node* temp = front;
front = front -> next;
rear -> next = front;
Printf ("%d dequeued. \n", temp -> data);
free (temp);
}
void display()
{
    if (front == NULL)
    { Printf ("Queue is empty. \n");
      return;
    }
struct Node *temp = front;
Printf ("Queue elements: ");
do
{
Printf ("%d", temp -> data);
temp = temp -> next;
}
while (temp != front);
Printf ("\n");
}
int main() {
int choice, value;
do {
Printf ("\n1. Enqueue 2. Dequeue \n3. Display \n
4. Find \n5. Exit
scanf ("%d", &choice);
```

```
switch (choice)
{
case 1:
printf ("Enter value:");
scanf ("%d", &value);
enqueue (value);
    break;
case 2:
    dequeue ();
    break;
case 3:
    display ();
    break;
case 4:
printf ("Exiting... \n");
    break;
default:
printf ("Invalid choice. \n");
}
}
while (choice != 4);
return 0;
}
```

output:

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter choice: 1

Enter value: 10

10 enqueued

1. Enque
2. Deque
3. Display
4. Exit.

Enter choice: 1

Enter value: 20

20 enqueued

1. Enqueue
2. Dequeue
3. Display
4. Exit.

Enter choice: 3

Que Elements: 10 20

1. Enqueue
2. Dequeue
3. Display
4. Exit.

Enter choice: 2

10 dequeued

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter choice: 4

Exiting....

(13) Implementation and Traversal (In-order, Pre-order, Post-order) of a Binary Search Tree

7/10/2026

Program:-

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node *left, *right;
};
struct Node * CreateNode(int value) {
    struct Node *newNode = (struct Node*) malloc(sizeof(struct Node));
    newNode->data= value;
    newNode->left = newNode->right= NULL;
    return newNode;
}
struct Node *insert (struct Node* root, int value) {
    if (root= NULL)
        return CreateNode(value);
    if(value < root->data)
    root->left = insert (root->left, value);
    else if (value > root->data)
        root->right =insert (root->right, value);
    return root;
}
void inorder (struct Node *root) {
    if(root== NULL) return;
    inorder (root->left);
    Printf ("%d", root->data);
    inorder (root->right);
```

```
Void preorder (struct Node * root) {
    if (root == NULL) return;
   Printf ("%d", root -> data);
    Preorder (root -> left);
     Pre order ( root -> right);
  }
Void postorder (struct Node * root) {
     if (root == NULL) return;
      Postorder (root -> left);
       Post order ( root -> right);
      Printf ("%od", rool -> data);
}
int main() {
       struct Node * root = NULL;
        int n, value;
        Printf ("Enter number of nodes:");
         scanf ("%od", &n);
       Printf ("Enter %d values:", n);
       for (int i=0; i<n; i++) {
           Scanf ("%d", &value);
         root = insert (root, value);
      }
Printf ("In in-order traversal:");
 inorder (root);
Printf ("In pre-order traversal:");
  Preorder (root);
Printf ("In post-order traversal:");
  Postorder (root);
   Printf ("In");
   return 0;
```

Sample output:-

Input:-

Enter number of nodes: 7

Enter 7 values: 50 30 70 20 40 60 80

output:-

In-order traversal: 20 30 40 50 60 70 80

Pre-order traversal: 50 30 20 40 70 60 80

Post-order traversal: 20 40 30 60 80 70 50.

## (11) Graph search Algorithms (BFS & DFS)

Aim:-

To write a C program to represent a graph using adjacency matrix and traverse it using Breadth-first search (BFS) and Depth-First Search (DFS).

Algorithm:-

1. Read the number of vertices and edges, and construct the adjacency matrix representing the graph (mark adj[u][v] = adj[v][u] = 1 for each edge, since the graph is undirected).

2. BFS: mark the starting vertex as visited and insert it into a queue; while the queue is not empty, remove a vertex, print it, and insert all its unvisited adjacent vertices into the queue, marking them as visited.

3. DFS: mark the starting vertex as visited and print it; recursively visit each unvisited adjacent vertex in the same manner.

4. Display the sequence of vertices visited by BFS and by DFS starting from the given vertex.

Program:-

```
#include <stdio.h>
#define MAX 20

int adj[MAX][MAX];
int visited[MAX];
int n; /* number of vertices */

void bfs(int start) {
  int queue[MAX], front=0, rear=0;
  int visitedLocal[MAX] = {0};
  visitedLocal[start]=1;
  queue[rear++]=start;
    Printf("BFS traversal:");
  while (front<rear) {
   int current = queue[front++];
   Printf("%d", current);
  for (int i=0; i<n; i++)
{
    if (adj[current][i]=1 && !visitedLocal[i])
  {  visitedLocal[i]=1;
     queue[rear++]=i;
    }
   }
  }
   Printf("\n");
}
```

```
void dfs (int start) {

    for(int i=0; i<n; i++)
      visited [i] =0;
      Printf ("Dfs traversal :");
      dfs Util (start);
       Printf ("\n");

}
int main() {
      int edges, u, v, start;
      Printf ("Enter number of vertices :");
      scanf ("%d", &n);
      Printf (" Enter number of edges : ");
      scanf ("%d", & edges);
      Printf ("Enter edges (u v), vertices numbered 0 to %d : \n", n-1);
for (int i=0; i< edges ; i++)
  {   scanf ("%d %d", &u, &v);
  adj [u][v] = 1;
      adj [v][u] = 1; /* undirected graph */
  }
Printf ("Enter starting vertex :");
  scanf ("%d", &start);

   bfs (start);
   dfs ( start);
   return 0;
```

## Sample Input/output

Input:-

Enter number of vertices: 5

Enter number of edges: 5

Edges: 0-1, 0-2, 1-3, 2-4, 3-4.

Enter starting vertex: 0

output:-

BFS traversal: 0 1 2 3 4

DFS traversal: 0 1 3 4 2

Result:-

The C program to traverse a graph using BFS and DFS was implemented and executed Successfully, and the output was verified.

(12) Implementation of Bubble, Selection and Heap Sorts.

21/10/26

Aim:-

To write a C program to sort a given set of elements using Bubble sort, selection sort, and Heap sort techniques.

Algorithm:-

1. Bubble sort: repeatedly traverse the array, comparing each pair of adjacent elements and swapping them if they are in the wrong order; repeat for n-1 passes so the largest unsorted element "bubbles" to its correct position each pass.
2. Selection sort: for each position i from 0 to n-2, find the index of the minimum element in the unsorted portion arr[i..n-1] and swap it with arr[i]
3. Heap sort: Build a max-heap from the array using the heapify procedure; repeatedly swap the root (max. element) with the last unsorted element; reduce the heap size by one, and re-heapify the root.
4. Read the array elements, apply the chosen sorting technique and display the sorted array

Program:-

```
#include <stdio.h>
void printArray (int arr[], int n) {
    for (int i=0; i<n; i++)
        Printf ("%d", arr[i]);
        Printf("\n");
    }
void bubbleSort (int arr[], int n) {
    for (int i=0; i<n-1; i++) {
        for (int j=0; i<n-1; j++) {
            if (arr[j]>arr[j+1]) {
                int temp= arr[j];
                arr[j]= arr[j+1];
                arr[j+1]= temp;
            }
        }
    }
}
Void selectionSort (int arr[], int n) {
    for (int i=0; i<n-1; i++) {
        int minIdx=i;
        for (int j=i+1; j<n; j++) {
            if (arr[j]< arr[minIdx])
                minIdx=j;
            }
```

```
        int temp= arr[minIdx];
        arr[minIdx]= arr[i];
        arr[i]= temp;
    }
  }
Void heapify (int arr[], int n, int i) {
    int largest = i;
    int left = 2*i+1;
    int right = 2*i+2;

if (left < n && arr[left] > arr[largest])
        largest= left;
if (right < n && arr[right] > arr[largest])
    largest= right;
if (largest != i) {
        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;
        heapify (arr, n, largest);
    }
}
void heapsort (int arr[], int n){
  for (int i= n/2-1 ; i >=0; i--)
    heapify (arr, n, i);
```

```
for (int i = n-1; i>0 ; i--)
  } int temp = arr[0];
      arr[0] = arr[i];
      arr[i] = temp;
      heapify (arr, i, 0);
    }
  }
int main() {
    int n, choice;
  int Original[100], arr[100];
  Printf("Enter number of elements:");
   Scanf("%d", &n);
Printf("Enter %d elements:", n);
   for (int i=0; i<n; i++)
     Scanf("%d", &original[i]);
Printf("\n 1. Bubble Sort \n 2. Selection sort \n 3. Heapsort \n Ent
Scanf("%d", &choice);
  for (int i=0; i<n; i++)
      arr[i] = original[i];
     switch (choice) {
        case 1: bubble sort (arr, n); break;
        case 2: selection sort (arr, n); break;
        case 3: heapsort (arr, n); break;
        default: printf("Invalid choice.\n"); return 1;
  }
Printf("Sorted array:");
PrintArray(arr, n);
  return 0;
}
```

Sample Input / output :-

Input:-

Enter no. of elements: 6

Enter 6 elements: 64 25 12 22 11 90

Choice : 1 (Bubble sort) / 2 (selection sort) / 3 (Heap sort)

output:-

Sorted array: 11 12 22 25 64 90

(Same sorted output is produced by Bubble sort, selection sort and Heap sort)

Result:-

The C programs to sort an array using Bubble sort, selection sort, and Heap sort were verified.

(13) Implementation of Hashing with Collision Resolution using Linear probing

21/10/26

Aim:-

To write a C program to implement a hash table using the division hash function h(key) = key % TABLE_SIZE, and resolve collisions using linear probing.

Algorithm:-

1. Create a hash table of fixed size TABLE_SIZE and initialize all slots to EMPTY.
2. To insert a key: Compute its home index using the hash function h(key) = key % TABLE_SIZE.
3. If the slot at the home index is occupied (collision), probe linearly (index = (index + 1) % TABLE_SIZE) until an empty slot is found, then insert the key there; if the table becomes full before an empty slot is found, report overflow.
4. To search for a key: Compute its home index, then probe linearly from that index, comparing each occupied slot's value to the key, until the key is found or an empty slot/the starting index is reached again.
5. Display the contents of the hash table, showing the index and the value (or EMPTY) stored at each slot.

Program:-

```
#include <stdio.h>
#define TABLE_SIZE 10
#define EMPTY -1

int hashTable[TABLE_SIZE];

void initTable() {
for (int i=0; i<TABLE_SIZE; i++)
    hashTable[i] = EMPTY;
}

int hashFunction(int key) {
    return key % TABLE_SIZE;
}

void insert(int key) {
    int index = hashFunction(key);
    int startIndex = index;
    int probes = 0;

while (hashTable[index] != EMPTY) {
    index = (index+1) % TABLE_SIZE;
    probes++;
    if (index == startIndex)
    { printf("Hash table is full. Cannot insert %d.\n", key
        return;
    }
}
hashTable[index] = key;
printf("Key %d inserted at index %d
                (Probes: %d).\n", key, index, probes
}
```

```
int search (int key) {
    int index = hash function (key);
    int start Index = index;
while (hash Table [index] != EMPTY) {
  if (hash Table [index] == key)
    return index;
  index = (index+1) % TABLE_SIZE;
  if (index == start Index)
      break;
  }
  return -1;
  }
void display () {
    Printf ("In Index \t Values In");
  for (int i=0; i< TABLE_SIZE; i++) {
    if (hash Table [i] == EMPTY)
      Printf ("%d \t EMPTY \n", i);
    else
      Printf ("%od \t %d\n", i, hash Table [i]);
  }
}
int main () {
int n, key, choice, result;
  init Table ();
```

```
Printf("Enter no of keys to insert:");
scanf("%d",&n);
Printf("Enter %d keys:",n);
for(int i=0; i<n; i++)
{
    scanf("%d",&key);
    insert(key);
}
display();
do {
    Printf("\n1. search \n 2. Insert\n 3. Display\n 4. Exit\n
Enter ch
    scanf("%d",&choice);
    switch(choice) {
    case 1:
    Printf("Enter key to search:");
    scanf("%d",&key);
    result = search(key);
    if(result != -1)
        Printf("key %d found at index %d:\n",key,result);
    else
        Printf("key %d not found:\n",key);
    break;
    case 2:
    Printf("Enter key to insert:");
    scanf("%d",&key);
    insert(key);
    break;
    case 3:
    display();
    break;
```

```
case 4:
Printf("Exiting....\n");
   break;
default:
   Printf("Invalid choice.\n");
   }
Fwhile (choice != 4);
   return 0;
  }
```

**Sample Input/ output:-**

Input:-

Enter no.of keys to insert: 5

Enter 5 keys: 15 25 35 5 10

choice: 1 (search) -> 25.

Output:-

Key 15 inserted at index 5 (Probes: 0)

Key 25 inserted at index 6 (Probes: 1)

Key 35 inserted at index 7 (Probes: 2)

Key 5 inserted at index 8 (Probes: 3)

Key 10 inserted at index 0 (Probes: 0)

| Index | value. |
|---|---|
| 0 | 10 |
| 1 | EMPTY |
| 2 | EMPTY |
| 3 | EMPTY |
| 4 | EMPTY |
| 5 | 15 |
| 6 | 25 |
| 7 | 35 |
| 8 | 5 |
| 9 | EMPTY |

Key 25 found at index 6.

Result:-